InvestVerdict

Monthly Holdings — Complete Guide

Har mahine naye AMC ka portfolio data Supabase mein daalne ka poora tareeka — shuru se aakhir tak. Non-coder ke liye, aaram se samajhne ke liye. Saara code andar diya hai; kuch likhna nahi, bas copy-paste.

InvestVerdict Fund Analytics Engine

Kya-kya hai is guide mein

1 · Ye guide kya hai

Humne ek **website (dashboard)** banaya hai jo mutual funds ka data dikhata hai — kaun sa fund kya rakhta hai, kis stock ko sabse zyada AMCs ne khareeda, returns, comparison, sab. Ye data har AMC (fund house) ki **monthly portfolio file** se aata hai.

Har mahine AMCs nayi file publish karte hain. Tumhara kaam sirf itna hai: **wo nayi file uthao aur database (Supabase) mein daal do.** Ye guide wahi sikhaati hai — bina coding jaane.

> **Achhi khabar:** tumhe code samajhne ki zaroorat nahi. Sab code pehle se ready hai (aakhir mein diya hai). Tum bas **copy-paste** karoge, ek-do jagah **date ya link badloge**, aur **Run** dabaoge. Bas.

> **Ek baat yaad rakho:** ye guide sirf **holdings** (kaun sa fund kya rakhta hai) ke baare mein hai. NAV aur returns website khud internet se live le leti hai — uske liye kuch nahi karna.

2 · Zaroori cheezein — pehle ye samajh lo

Do tools use honge

1. Google Colab — ye ek website hai (colab.research.google.com) jahan tum code "cells" mein paste karke **Run** dabate ho. Socho ise ek notebook jaisa jahan har box (cell) ka apna kaam hai. Internet se files download karke Supabase mein bhejne ka kaam yahin hoga.

2. Supabase — ye tumhara **database** hai (jahan saara data rehta hai). Iska ek "SQL Editor" hota hai jahan hum ek command chala ke data ko website ke liye taiyaar karte hain.

Do "chaabiyan" (keys) — inhe mat mix karna

Key	Kya hai	Kahan use
service_role key	Poori power wali chaabi (data likh/mita sakti hai)	Sirf Colab code mein (`SUPABASE_KEY` ki jagah)
anon key	Sirf-padhne wali chaabi	Website (`index.html`) ke andar — usse haath mat lagao

service_role key laane ke liye: Supabase → apna project → Settings → API → "service_role" (secret) → copy.

Bahut zaroori — do baatein:
• service_role key **kisi ko mat do, kahin public mat daalo.** Ye poora database mita sakti hai.
• Har AMC ka ek **chhota naam (code)** hota hai — jaise `HDFC` , `Kotak` , `Invesco` . Ye naam **har mahine bilkul same** rakhna. Agar galti se `HDFC` ki jagah `HDFC MF` likh diya, to database mein **do alag fund houses** ban jaayenge aur saare numbers aadhe- aadhe bant jaayenge.

Project ki fixed jaankari

Supabase project	`ulunrpbayvlazzxpqrpj` ("MF Analyzer")
URL (code mein pehle se hai)	`https://ulunrpbayvlazzxpqrpj.supabase.co`
Website	ek hi `index.html` — data daalne se ye kabhi nahi badalti

3 · Har mahine ka routine (yahi 5 step har baar)

1. **Colab kholo aur parser chalao**
   colab.research.google.com par jao → New notebook. Pehle cell mein **Appendix A (parser.py)** ka poora code paste karo aur Run dabao. Koi output nahi aayega — ye bas 'taiyaari' hai. **Ye har baar sabse pehle chalana hai.** Agar Colab band/restart ho jaye, dobara chalao.

2. **Jis AMC ka data daalna hai, uska loader chalao**
   Naya cell banao, us AMC ka code paste karo (Appendix B/C/D mein diya hai), apni **service_role key** daalo, aur us mahine ke hisaab se **link ya date badlo** (Section 4 mein har AMC ke liye likha hai kya badalna hai). Phir Run dabao.

3. **Output dhyaan se dekho**
   Loader batayega: kitne schemes mile, kitne 100% pe jud rahe hain, koi file skip to nahi hui. Aakhir mein **'Pushed'** likha aaye — matlab data database mein chala gaya.

4. **Supabase mein 'after_load.sql' chalao — ek baar**
   Us mahine ke saare AMC push karne ke baad, Supabase → SQL Editor mein **Appendix E (after_load.sql)** ka poora code paste karke Run karo. Ye naye funds ko website ke liye taiyaar karta hai. **Ye chalaye bina naye funds website par nahi dikhenge.**

5. **Check karo**
   Section 5 ki query chala ke confirm karo har AMC sahi aaya. Bas — ho gaya.

Purana mahina daalna safe hai. July daalne se June ka data kharab nahi hota. Balki kisi fund ka doosra mahina daalne se website ka **"Changes"** tab zinda ho jaata hai (kya khareeda/becha).

Optional: "Leaders" tab aur AMC-median returns ke liye kabhi-kabhi **Appendix F (nav_monthly_ingest.py)** chala dena, phir Supabase mein: `refresh materialized view mv_fund_returns; analyze mv_fund_returns;` Compare ke returns ko iski zaroorat nahi — wo live aate hain.

4 · Har AMC — tarika, link, aur kya badalna hai

Char tarike hain. Har AMC kaunsa use karta hai aur **har mahine tum sirf ek cheez badalte ho** — neeche card mein saaf likha hai.

METHOD A · DIRECT LINK → LOAD.PY

Bank of India (code: `BOI`)

boimf.in → Investor Corner → Monthly Portfolio se `.xlsx` link copy karo.

> **Har mahine sirf ye badlo:** load.py ke `SOURCES` mein BOI ki line ka **link** (usme mahine ka naam).

June-2026 ke actual links (agli baar month badal dena):

```
BOI    https://www.boimf.in/docs/default-source/investorcorner/monthly-portfolio/monthly-portfolio---30-june-2026.xlsx?sfvrsn=ea175794_3
```

METHOD A · DIRECT LINK → LOAD.PY

PPFAS (Parag Parikh) (code: `PPFAS`)

amc.ppfas.com → Downloads → Portfolio Disclosure → year folder se `.xls` link.

> **Har mahine sirf ye badlo:** load.py ke `SOURCES` mein PPFAS ki line ka **link**.

June-2026 ke actual links (agli baar month badal dena):

```
PPFAS  https://amc.ppfas.com/downloads/portfolio-disclosure/2026/PPFAS_Monthly_Portfolio_Report_June_30_2026.xls?08072026_1
```

METHOD A · DIRECT LINK → LOAD.PY (YA FILE UPLOAD)

Kotak (code: `Kotak`)

Kotak MF site → Consolidated SEBI Portfolio ka `.xlsx` link. Humne ise file download karke **upload** se bhi kiya tha (Method D).

> **Har mahine sirf ye badlo:** load.py ke `SOURCES` mein Kotak ki line ka **link** (usme date).

June-2026 ke actual links (agli baar month badal dena):

```
Kotak  https://vatseelabs-s3.kotakmf.com/FormsDownloads/Portfolios/Consolidated-SEBI-Portfolio-as-on-June-30,-2026/ConsolidatedSEBIPortfolioJune2026.xlsx
```

METHOD B · API SCRAPER → INVESCO.PY

Invesco (code: `Invesco`)

Invesco ki site JavaScript se banti hai (link copy nahi hota) aur har scheme ki alag file deti hai. Isliye hum unki apni API se seedha uthate hain — tumhe kuch dhundhna nahi, code khud kar leta hai.

> **Har mahine sirf ye badlo:** invesco.py mein sirf `MONTH = "Jul"` (teen akshar) aur January mein `YEAR`. Baaki kuch nahi.

METHOD C · SERVER ACTION → JIOBLACKROCK.PY

JioBlackRock (code: `JioBR`)

Jio ki site bhi JavaScript se banti hai. Hum unki 'server action' call karke file list uthate hain (code khud karta hai).

> **Har mahine sirf ye badlo:** jioblackrock.py mein sirf `TARGET = "2026-07"` (YYYY-MM), ya `None` chhod do to latest apne aap aa jaata hai.

Method D — koi bhi AMC ki Excel file, chahe structure kaisa bhi ho

METHOD D · FILE UPLOAD → LOAD.PY

Koi bhi doosri AMC (file download karke) (code: `<CODE>`)

Jis AMC ka file tum khud download karte ho (koi saaf link na ho): load.py mein `SOURCES = []` chhod do, `DEFAULT_AMC = "<CODE>"` set karo, Run karo, aur upload box mein file(s) drag kar do.

Har mahine sirf ye badlo: sirf `DEFAULT_AMC` (us AMC ka code).

Har AMC ki file ka layout alag hota hai — par tumhe kuch code nahi likhna. Parser column ko uske **naam** (Instrument / ISIN / % to NAV / Market Value) se dhundhta hai, position se nahi. Isliye ek-file-per-scheme ho ya consolidated workbook — sab same load.py se chal jaate hain. Kotak humne isi upload tareeke se kiya tha.

Jo AMCs pehle se database mein hain (parser inhe pehchanta hai)

ABSL, HDFC, SBI, ICICI, Nippon, Kotak, DSP, Motilal, Bandhan, Mirae, Axis, Tata, Quant, Quantum, Abakkus, 360ONE — **+ Invesco, BOI, PPFAS, JioBR**. Inme se kisi ko refresh karna ho to wahi tarika: direct link → load.py; hand file → load.py upload.

Abhi tak load nahi hue (jab chaho tab BOI/PPFAS jaise kar lena): HSBC (humne abhi chhoda tha), aur UTI, Franklin, Canara Robeco, Sundaram, Edelweiss, LIC, Baroda BNP, Mahindra Manulife. Sab monthly portfolio publish karte hain — link uthao aur BOI/PPFAS/Kotak jaise load kar do, kuch naya likhne ki zaroorat nahi.

5 · Check kaise karein — sahi hua ya nahi

after_load.sql chalane ke baad, Supabase SQL Editor mein ye chalao. Ek nazar mein pata chal jaata hai har AMC aaya ya nahi, aur website par dikhega ya nahi:

```
select h.amc,
       count(distinct h.scheme_name)                          as schemes,
       count(*)                                               as rows,
       max(h.portfolio_date)                                  as latest,
       coalesce(fp.searchable, 0)                             as searchable,
       coalesce(fs.in_engine, 0)                              as in_analytics
from mf_holdings h
left join (select amc, count(*) searchable from v_fund_picker  group by amc) fp on fp.amc = h.amc
left join (select amc, count(*) in_engine  from mv_fund_stats  group by amc) fs on fs.amc = h.amc
group by h.amc, fp.searchable, fs.in_engine
order by h.amc;
```

- **schemes** 0 se zyada aur **latest** = jo mahina daala → push sahi hua.
- **searchable** aur **in_analytics** ≈ schemes → after_load.sql chal gaya, website par aa gaya.
- dono **0** → after_load.sql chalana bhool gaye, chala do.

> **Scheme count ka andaaza (galat file pakadne ke liye):** Invesco ~44, BOI ~24, PPFAS ~7, Kotak ~55–60, JioBR ~7–10. Agar 50 ki jagah 5 aaye, to galat file hai — **push mat karna.**

AUM sahi hai ya nahi (poore database ka): ye ₹60–65 lakh crore ke aas-paas hona chahiye.

```
select round(sum(aum_cr)) as total_aum_cr from mv_fund_stats;
```

6 · Jo galtiyan humne ki (aur ab theek hain)

Ye isliye likha hai taaki agar tumhe kabhi koi purani baat sunai de ya koi message dikhe, tum samajh sako ki wo kyun tha aur ab handle ho chuka hai. **Tumhe kuch karna nahi** — sab guards code mein daal diye hain.

Kya galti hui thi	Ab kya guard hai
Colab Secrets kaam nahi kar raha	Key ab seedha code mein daali jaati hai (SUPABASE_KEY).
Galat project ki key (401 error push pe)	Har loader pehle key check karta hai — galat project/role pe turant ruk jaata hai.
'No module named supabase'	Har loader pehli line par khud install kar leta hai.
'parse_workbook not defined'	Parser cell (Appendix A) nahi chalaya — loader ab bata deta hai, pehle wo chalao.
Aadha data aaya (1000 rows ki limit)	Bade reads ab tukdo mein (paging) hote hain.
Per-scheme files ek doosre ko mita rahe the	Sab files jodkar ek hi baar push hoti hain (Invesco jaise).
'Nil' wali khaali headings holding ban rahi thi	Ab value ko number hona zaroori — parser v6.
AUM 4.8% zyada dikh raha tha	Negative Net Current Assets ab shamil (AMC bhi ginta hai).
advisorkhoj ne galat file di (5 scheme)	Hamesha scheme count check karo push se pehle.
JavaScript site pe link nahi milta (Invesco/Jio)	Unki API/server-action use karte hain, link guess nahi.
Excel download khali aa raha tha	Ab ek hi sheet mein poora data, numbers asli numbers.
Compare mein 'no NAV history'	Compare ab returns live internet se leta hai — har fund chalta hai.

7 · Kuch atak jaye to

- **Loader 'WRONG PROJECT' ya role error deta hai** → galat key. Supabase → Settings → API se **service_role** key dobara copy karo.
- **File parse nahi ho rahi / 0 rows** → link purana ho gaya hoga. Loader "served HTML, not a workbook" bolega — us AMC ka naya link lo.
- **Naye funds website par nahi dikh rahe** → after_load.sql chalana bhool gaye.
- **Changes tab kisi house ke liye khali** → us house ka sirf ek mahina hai; uska ek purana mahina bhi load kar do.
- **Samajh na aaye** → mujhe (Claude) output ka screenshot bhej dena, main bata dunga.

Appendix A · parser.py — sabse pehle chalao (har session)

Ye 'taiyaari' hai. Colab ke pehle cell mein poora paste karo, Run karo. Koi output nahi aata. Har session mein ek baar (Colab restart hone par phir se).

```
"""
AMC portfolio-disclosure parser -- v6

Columns are matched by HEADER TEXT, not position, so one parser handles every
AMC's layout. Paste as one Colab cell; parse_workbook() and push() are what the
loaders call.

v5 changes one thing over the v4 described in MF_PROJECT_HANDOFF_PART2.md:
BOILER now catches objective lines, so the scheme name is read correctly for
the ~19 schemes that were carrying an objective sentence or a sheet code
instead of a name (Quant 7, Tata 6, Motilal 4, SBI 2). Those were previously
patched through scheme_alias, which fixed the mapping but left the ugly string
showing everywhere the AMC's own name is displayed.

Re-parse and re-push Quant, Tata, Motilal and SBI to pick this up. Everything
else is unchanged, so other AMCs do not need reloading.

v6 stops empty section headings becoming holdings. An AMC writes "Nil" beside a
category it holds nothing in ("Term Deposits  Nil"), and pd.notna("Nil") is
True, so the row read as a position -- KEEP_NO_ISIN's deposit/margin patterns
then rescued it from the heading branch. A value now has to parse as a NUMBER.
That produced 1,606 phantom rows across Tata, SBI, ICICI and Quantum; they carry
no ISIN, no weight and no value, so nothing downstream moves when they go.
"""

# !pip -q install requests pandas openpyxl xlrd supabase

import io, re, requests, pandas as pd

ISIN_RE = re.compile(r"^[A-Z]{2}[A-Z0-9]{9}[0-9]$")
SKIP_RE = re.compile(r"^(sub\s*total|total|grand\s*total|net\s*assets\s*$|notes?\s*:|less\s*:)",
re.I)
STOP_RE = re.compile(r"^(grand\s*total|hedging position|other than hedging|"
                     r"derivative (disclosure|position)|disclosure regarding derivative|"
                     r"total exposure|notes?\s*:|riskometer|portfolio classification|"
                     r"navs? per unit|dividend history|bonus history)", re.I)

# TOP-LEVEL banners only. "Government Securities" is a sub-heading inside
# DEBT INSTRUMENTS -- matching it here flattens the hierarchy.
SECTION_RE = re.compile(r"^(equity\s*[&a]|equity shares|equity related|debt instrument|"
                        r"money market|others?\s*$|derivativ|units? issued|foreign securit|"
                        r"mutual fund unit|cash\s*[&a]|gold\s*$|silver\s*$|treps|reits|invits)",
re.I)

# A row with a value but no ISIN/quantity is usually a heading carrying a
# sub-total. Default is KEEP; only drop when the label reads as a heading.
HEADING_RE = re.compile(
    r"^(equity|debt|money market|others?\b|derivativ|units? issued|foreign|gold\b|silver\b|"
    r"mutual fund unit|reits?\b|invits?\b|cash\s*&|listed|unlisted|privately|securiti[sz]|"
    r"\(?[a-z]\)\s|commercial paper|certificate of deposit|treasury bill|"
    r"government securit|state government|central government|non[- ]convertible|"
    r"zero coupon|strips\b|bills re|alternative investment|preference share|"
    r"equity share|exchange traded fund|^bond|corporate debt|floating rate note|"
```

```
    r"fixed deposit|term deposit|units? of|^cds?$|^cp$|^deposits?\b|"
    r"^t[- ]?bills?|^ncds?$|^psu\b|^ptc\b|.*\btotals?\s*$)", re.I)

# Real holdings that legitimately carry no ISIN.
KEEP_NO_ISIN = re.compile(r"treps|tri-?party|reverse repo|net current asset|net receivable|"
                          r"net payable|cash\s*(&|and)|margin|deposit|corporate debt repo|net
asset", re.I)

# Text that must never be mistaken for a scheme name.
#
# The last alternation is the v5 addition. Several AMCs put the scheme's
# investment objective on the row below the name, and it reads enough like a
# fund name to win: "Multi Cap Fund - An open ended equity scheme investing
# across large cap, mid cap, small cap stocks" was what Quant's schemes were
# being called. Tata writes "Investment in equity and equity related
# instruments comprised in ..." and "TRSF-CONSERVATIVE PLAN: * ...".
BOILER = re.compile(r"portfolio statement|figures as on|as on\s*:|back to index|registered
office|"
                    r"^cin\b|e-?mail|asset management company|investment manager|^index$|"
                    r"disclosure|^notes?\b|mutual fund$|^scheme name|fund size|"
                    r"^an?\s+(open|close|closed)[\s\-]*end|^an?\s+open-?\s*end|"
                    r"risk-?o-?meter|riskometer|product is suitable|investors should|"
                    r"suitable for investors|please consult|^\s*[•·▪◦]|"
                    r"^investment in (equity|debt|securities)|^trsf|^an open ended|"
                    r"^this product|^the scheme", re.I)

LOOKS_FUND = re.compile(r"\b(fund|etf|scheme|plan|yojana|series|fof|fmp|elss|saver|index|"
                        r"advantage|opportunit|savings)\b", re.I)

DATE_RE = re.compile(r"(\d{1,2}[-/\s][A-Za-z]{3,9}[-/,\s]{1,2}\d{2,4}"
                     r"|[A-Za-z]{3,9}\s+\d{1,2},?\s*\d{4}"
                     r"|\d{1,2}[-/]\d{1,2}[-/]\d{2,4}|\d{4}-\d{2}-\d{2})")

FIELDS = [("isin", r"^\s*isin"),
          ("instrument_name", r"name of\s+(the\s+)?
instrument|company\s*/\s*issuer|instrument\s*/\s*issuer"),
          ("coupon_pct", r"^\s*coupon"),
          ("industry_rating", r"industry|rating"),
          ("quantity", r"^\s*quantity|^\s*qty"),
          ("market_value_lacs", r"market\s*/?\s*fair\s*value|market\s*value|exposure/?
market|mkt\.?\s*val"),
          ("pct_to_nav", r"%\s*to\s*(nav|aum|net\s*assets)"),
          ("yield_pct", r"^\s*(yield|ytm)")]
NUMERIC = ["coupon_pct", "quantity", "market_value_lacs", "pct_to_nav", "yield_pct"]

_norm = lambda v: re.sub(r"\s+", " ", str(v)).strip().lower()

def _num(v):
    if v is None or (isinstance(v, float) and pd.isna(v)): return None
    return pd.to_numeric(str(v).replace(",", "").replace("%", "").strip(), errors="coerce")

def _map_columns(row):
    cells = {c: _norm(v) for c, v in enumerate(row) if pd.notna(v)}
    colmap, taken = {}, set()
    for field, pat in FIELDS:
        for c in sorted(cells):
            if c in taken or field in colmap: continue
            if re.search(pat, cells[c]): colmap[field] = c; taken.add(c)
    return colmap

def _clean(t):
    t = re.sub(r"\s+", " ", str(t)).strip()
    return re.sub(r"\s*\((?=.{25,}).*", "", t).strip()
```

```
def _clean_name(t):
    t = re.sub(r"^portfolio of\s*", "", str(t), flags=re.I)
    t = re.sub(r"\s+as on\s+.*$", "", t, flags=re.I)
    t = re.sub(r"\s*-\s*an?\s+(open|close|closed)[\s\-]*end.*$", "", t, flags=re.I)
    return re.sub(r"\s+", " ", t).strip()

def parse_sheet(d, sheet=""):
    hdr = colmap = None
    for i in range(min(30, len(d))):
        cm = _map_columns(d.iloc[i].values)
        if {"isin", "instrument_name", "pct_to_nav"} <= set(cm): hdr, colmap = i, cm; break
    if hdr is None: return pd.DataFrame()

    c_name, c_pct = colmap["instrument_name"], colmap["pct_to_nav"]
    c_mv = colmap.get("market_value_lacs")
    c_isin, c_qty = colmap["isin"], colmap.get("quantity")

    scheme, cands = None, []
    for i in range(hdr):
        for v in d.iloc[i].values:
            if pd.isna(v): continue
            t = _clean(v)
            if 6 < len(t) < 160 and not BOILER.search(t):
                cands.append(t)
                if LOOKS_FUND.search(t): scheme = t
    scheme = _clean_name(scheme or (max(cands, key=len) if cands else sheet))

    as_on = None
    for i in range(min(hdr + 1, len(d))):
        # merged cells repeat a row's text; collapse before matching or a greedy
        # capture grabs "30-Jun-2026 Portfolio"
        t = " | ".join(dict.fromkeys(str(v) for v in d.iloc[i].values if pd.notna(v)))
        if not re.search(r"as on|as at|figures as|period ended|month ended|as of", t, re.I):
continue
        for m in DATE_RE.finditer(t):
            raw = re.sub(r",(?=\S)", ", ", m.group(1).strip())
            as_on = pd.to_datetime(raw, dayfirst=bool(re.match(r"^\d{1,2}[-/]", raw)),
errors="coerce")
            if pd.notna(as_on) and as_on.year > 1990: break
            as_on = None
        if as_on is not None: break

    rows, section, sub = [], None, None
    for i in range(hdr + 1, len(d)):
        r = d.iloc[i]
        name = r.iloc[c_name] if c_name < len(r) else None
        pct = r.iloc[c_pct] if c_pct < len(r) else None
        mv = r.iloc[c_mv] if c_mv is not None and c_mv < len(r) else None
        # A value must be a NUMBER, not merely present. AMCs write "Nil" beside
        # an empty category ("Term Deposits  Nil"), and pd.notna("Nil") is True --
        # so the row looked like a holding, and KEEP_NO_ISIN's deposit/margin
        # patterns then rescued it from the heading branch below. That is how
        # 1,606 empty headings across Tata, SBI, ICICI and Quantum became rows.
        has_val = _num(pct) is not None and pd.notna(_num(pct)) \
               or _num(mv)  is not None and pd.notna(_num(mv))
        label = "" if pd.isna(name) else re.sub(r"\s+", " ", str(name)).strip()

        iv = r.iloc[c_isin] if c_isin < len(r) else None
        has_isin = pd.notna(iv) and bool(ISIN_RE.match(str(iv).strip().upper()))
        has_qty = c_qty is not None and c_qty < len(r) and pd.notna(r.iloc[c_qty])

        # HDFC/Nippon/Kotak write headings in a column other than the name one.
```

```
        # ISINs are skipped so Motilal's blank-name rows keep a NULL name.
        if not label and not has_val:
            for v in r.values:
                if pd.notna(v) and isinstance(v, str) and len(str(v).strip()) > 2:
                    cand = re.sub(r"\s+", " ", str(v)).strip()
                    if not ISIN_RE.match(cand.upper()):
                        label = cand
                    break

        if label and STOP_RE.match(label): break
        if label and SKIP_RE.match(label): continue
        # An ISIN is proof of a position, so such a row is never a heading --
        # even if its value cell is blank or unreadable, keep it and let the
        # null show rather than losing a holding silently.
        if label and not has_val and not has_isin:
            if SECTION_RE.match(label): section, sub = label, None
            else: sub = label
            continue
        if (has_val and not has_isin and not has_qty and label
                and HEADING_RE.match(label) and not KEEP_NO_ISIN.search(label)):
            if SECTION_RE.match(label): section, sub = label, None
            else: sub = label
            continue
        if not (has_val or has_isin) or not (label or has_isin): continue

        rec = {f: (r.iloc[c] if c < len(r) else None) for f, c in colmap.items()}
        rec["instrument_name"] = re.sub(r"[£^#*@~+\s]+$", "", label).strip() or None
        rec.update(section=section, sub_section=sub, sheet=sheet,
                   scheme_name=scheme, portfolio_date=as_on)
        rows.append(rec)

    if not rows: return pd.DataFrame()
    h = pd.DataFrame(rows)
    h["isin"] = h["isin"].astype(str).str.strip().str.upper()
    h.loc[~h["isin"].str.match(ISIN_RE, na=False), "isin"] = None
    for c in NUMERIC: h[c] = h[c].map(_num) if c in h.columns else None
    h["is_security"] = h["isin"].notna()
    return h

def parse_workbook(src, amc="", frequency="", source=""):
    xl, frames = pd.ExcelFile(src), []
    for sh in xl.sheet_names:
        if _norm(sh) in {"index", "notes", "disclaimer", "contents"}: continue
        try: d = xl.parse(sh, header=None)
        except Exception: continue
        h = parse_sheet(d, sheet=sh)
        if not h.empty: frames.append(h)
    if not frames: return pd.DataFrame()
    out = pd.concat(frames, ignore_index=True)
    # Some AMCs write 9.18, others 0.0918. Decide once per FILE from the median
    # scheme total -- arbitrage funds legitimately exceed 100, so per-scheme
    # detection misfires. Get this wrong and every number is 100x off silently.
    med = out.groupby("scheme_name")["pct_to_nav"].sum().median()
    if pd.notna(med) and 0.5 < med < 1.6: out["pct_to_nav"] *= 100
    out.insert(0, "amc", amc)
    out["frequency"], out["source_file"] = frequency, source
    return out


# ============================================================================
# Verification -- run this before pushing, every time
# ============================================================================
def verify(df, label=""):
```

```
    n_sch = df["scheme_name"].nunique()
    sums = df.groupby("scheme_name")["pct_to_nav"].sum()
    ok = sums.between(98, 102).sum()
    bad_dates = df["portfolio_date"].isna().sum()
    ugly = [s for s in df["scheme_name"].unique()
            if len(str(s)) > 90 or re.match(r"^[A-Z]{2,}\d|^[A-Z]{5,}$", str(s))]
    print(f"{label or df['amc'].iloc[0]}")
    print(f"  rows            {len(df):,}")
    print(f"  schemes         {n_sch}")
    print(f"  %NAV sums to 100 {ok}/{n_sch}")
    print(f"  NULL dates      {bad_dates}")
    print(f"  suspect names   {len(ugly)}")
    for s in ugly[:8]: print(f"      {str(s)[:100]}")
    return ok, n_sch


# ============================================================================
# Push -- delete-then-insert per (amc, portfolio_date, frequency)
# ============================================================================
def make_push(sb):
    import json
    COLS = ["amc", "scheme_name", "sheet_code", "portfolio_date", "frequency", "section",
            "sub_section", "instrument_name", "isin", "industry_rating", "coupon_pct",
            "quantity", "market_value_lacs", "pct_to_nav", "yield_pct", "is_security",
            "needs_review", "source_file"]

    def push(df):
        # groupby drops NaN keys, so a NULL date would insert zero rows and
        # report no error at all. Assert before touching the database.
        n = df["portfolio_date"].isna().sum()
        assert n == 0, f"{n} rows have NULL portfolio_date"

        up = df.copy().rename(columns={"sheet": "sheet_code"})
        s = up.groupby(["amc", "scheme_name"])["pct_to_nav"].transform("sum")
        up["needs_review"] = ~s.between(98, 102)     # arbitrage/hybrid: flag, never drop
        up["portfolio_date"] = pd.to_datetime(up["portfolio_date"]).dt.strftime("%Y-%m-%d")
        for c in COLS:
            if c not in up.columns: up[c] = None
        up = up[COLS]

        for key, g in up.groupby(["amc", "portfolio_date", "frequency"]):
            amc, dt_, freq = key
            sb.table("mf_holdings").delete() \
              .eq("amc", amc).eq("portfolio_date", dt_).eq("frequency", freq).execute()
            rows = json.loads(g.to_json(orient="records"))   # NaN->null, numpy->native
            for i in range(0, len(rows), 500):
                sb.table("mf_holdings").insert(rows[i:i + 500]).execute()
            print(f"  {amc:8s} {dt_} {freq:12s} {len(rows):6,} rows")
    return push
```

Appendix B · load.py — direct-link + file-upload loader (BOI, PPFAS, Kotak, koi bhi Excel)

Direct link wale AMC ke liye SOURCES mein link daalo; ya file khud upload karni ho to SOURCES khaali chhod ke DEFAULT_AMC set karo. service_role key daalna mat bhoolna.

```
"""
Universal loader  --  drag files in, they land in mf_holdings

One cell, any number of AMCs, any number of months, in one go. Replaces the
per-AMC scripts: the AMC and the disclosure frequency are read off the filename,
the portfolio date is read out of the workbook itself, and nothing is pushed
until it has passed the same checks you would have run by hand.

    Cell 1 : colab/parser.py     (paste once per Colab session)
    Cell 2 : this file           (paste once, run every time)
    Then   : sql/after_load.sql  in Supabase

WHAT IT DOES NOT DO, deliberately: it will not invent an AMC code. A house is
identified by a short code ('HDFC', 'ICICI', 'Nippon') that must match what is
already in the database exactly -- 'HDFC' and 'HDFC MF' would become two
different fund houses, and every cross-fund number would quietly split in half.
Unrecognised files are listed and skipped, never guessed at.

LOADING AN OLDER MONTH IS SAFE. push() deletes per (amc, portfolio_date,
frequency), so June does not disturb July. That is how you light up the Changes
tab: load any house's previous month and every one of its schemes gains history.
"""

# Self-installing, because a fresh Colab session has no supabase client and a
# commented-out !pip line is a trap. subprocess rather than !pip so this works
# whether or not the cell is being run by IPython.
import subprocess, sys
subprocess.run([sys.executable, "-m", "pip", "install", "-q",
                "requests", "pandas", "openpyxl", "xlrd", "supabase"], check=False)

import io, re, requests, pandas as pd
from supabase import create_client
from google.colab import files

SUPABASE_URL = "https://ulunrpbayvlazzxpqrpj.supabase.co"
SUPABASE_KEY = "PASTE_SERVICE_ROLE_KEY_HERE"      # service_role, not anon

# ---------------------------------------------------------------------------
# Key guard -- check the key BEFORE anything is downloaded or pushed.
#
# This has bitten twice: a key from a different Supabase project looks identical
# and fails with a bare "401 Invalid API key" only at the push, after every file
# has already been fetched and parsed. The project ref and the role both live in
# the JWT payload, so both are checkable up front without calling anything.
# ---------------------------------------------------------------------------
def check_key(url, key):
    import base64, json
    if key.startswith("PASTE_"):
        raise SystemExit("SUPABASE_KEY is still the placeholder. Paste the "
                         "service_role key from Settings > API.")
    try:
```

```
        body = key.split(".")[1]
        body += "=" * (-len(body) % 4)               # JWT strips the padding
        claims = json.loads(base64.urlsafe_b64decode(body))
    except Exception:
        raise SystemExit("SUPABASE_KEY is not a JWT. Copy it again, whole.")

    want = url.split("//")[1].split(".")[0]
    ref, role = claims.get("ref"), claims.get("role")
    print(f"key: role={role}  project={ref}  (expected {want})")

    if ref != want:
        raise SystemExit(f"WRONG PROJECT -- this key belongs to '{ref}', not "
                         f"'{want}'. Nothing was pushed.")
    if role != "service_role":
        raise SystemExit(f"This is the '{role}' key. Writing to mf_holdings "
                         "needs service_role -- anon is blocked by RLS.")


check_key(SUPABASE_URL, SUPABASE_KEY)

# parse_workbook, verify and make_push come from parser.py, which has to be run
# as its own cell first -- once per Colab session, again after any restart.
# Without this the failure is a bare NameError halfway down the file.
for _fn in ("parse_workbook", "verify", "make_push"):
    if _fn not in globals():
        raise SystemExit(
            f"'{_fn}' is not defined -- colab/parser.py has not been run in this "
            "session.\nPaste parser.py into its own cell, run it, then run this one.")

sb   = create_client(SUPABASE_URL, SUPABASE_KEY)
push = make_push(sb)                              # from parser.py

# ============================================================================
# 1. Which house is this file from?
#
# Order matters -- the first match wins, so longer and more specific patterns
# come first. 'sbi' would otherwise match inside 'hsbc'.
#
# To add a house: put the short code you want stored on the left and a pattern
# that appears in its filenames on the right. Keep the code SHORT and reuse it
# forever; it is the join key for every cross-fund query.
# ============================================================================
AMC_PATTERNS = [
    ("HSBC",    r"hsbc"),
    ("SBI",     r"\bsbi\b|all[- ]schemes"),
    ("ICICI",   r"icici|ipru"),
    ("HDFC",    r"hdfc"),
    ("Nippon",  r"nippon|nimf|reliance"),
    ("ABSL",    r"aditya|birla|absl"),
    ("Kotak",   r"kotak"),
    ("Axis",    r"axis"),
    ("DSP",     r"\bdsp\b"),
    ("Mirae",   r"mirae"),
    ("Bandhan", r"bandhan|idfc"),
    ("Tata",    r"tata"),
    ("Motilal", r"motilal|\bmo\b|moamc"),
    ("Quantum", r"quantum"),
    ("Quant",   r"quant"),            # after Quantum, or it swallows it
    ("Invesco", r"invesco"),
    ("UTI",     r"\buti\b"),
    ("Franklin", r"franklin|templeton"),
    ("Edelweiss", r"edelweiss"),
    ("PPFAS",   r"ppfas|parag"),
```

```
    ("Canara",  r"canara|robeco"),
    ("Sundaram", r"sundaram"),
    ("LIC",     r"\blic\b"),
    ("Baroda",  r"baroda|bnp"),
    ("JM",      r"\bjm\b"),
    ("Navi",    r"navi"),
    ("WhiteOak", r"whiteoak|white oak"),
    ("Samco",   r"samco"),
    ("Trust",   r"trustmf|trust mutual"),
    ("360ONE",  r"360|iifl"),
    ("Abakkus", r"abakkus"),
    ("Groww",   r"groww"),
    ("Zerodha", r"zerodha"),
    ("Bajaj",   r"bajaj"),
    ("Helios",  r"helios"),
    ("Old Bridge", r"old ?bridge"),
    ("Unifi",   r"unifi"),
]

# Filename says nothing useful? Map it by hand here: {"weird_name.xlsx": "HDFC"}
AMC_OVERRIDE = {}

# Set this when every file in this run belongs to one house and the filenames do
# not say so -- an AMC that publishes one file per scheme names them after the
# scheme ("largecap-fund-june-2026.xlsx"), and writing 60 AMC_OVERRIDE entries by
# hand is not a workflow. Leave as None to require the filename to identify it.
DEFAULT_AMC = None


def detect_amc(fname):
    if fname in AMC_OVERRIDE: return AMC_OVERRIDE[fname]
    low = fname.lower()
    for code, pat in AMC_PATTERNS:
        if re.search(pat, low): return code
    return DEFAULT_AMC


def detect_frequency(fname):
    low = fname.lower()
    if re.search(r"fortnight|15th|fn[- ]?portfolio", low): return "fortnightly"
    if re.search(r"half[- ]?year|halfyearly",        low): return "half-yearly"
    return "monthly"


# ============================================================================
# 2. Where the files come from -- URLs, uploads, or both
#
# URLs are the easier route where the AMC exposes one: nothing to download and
# re-upload, and re-running next month is a one-character edit. Paste them here.
# Leave the list empty to go straight to the upload box.
#
# These links change every month and some AMCs render their disclosure page in
# JavaScript, so there is no link to copy -- those you download by hand and
# drag in. Both paths run through exactly the same checks below.
# ============================================================================
SOURCES = [
    # "https://www.example-amc.com/monthly-portfolio-june-2026.xlsx",
]

UA = {"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
                    "AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126 Safari/537.36"}

uploaded = {}
```

```
for url in SOURCES:
    name = url.rsplit("/", 1)[-1].split("?")[0] or url
    try:
        r = requests.get(url, headers=UA, timeout=300)
        # An AMC that has moved the file usually serves an HTML error page with
        # status 200, which would then "parse" to zero rows and look like a
        # parser bug. Catch it here where the cause is obvious.
        if r.status_code != 200:
            print(f"  {name}: http {r.status_code} -- skipped"); continue
        if r.content[:200].lstrip()[:1] == b"<":
            print(f"  {name}: got HTML, not a workbook -- link is stale, skipped")
            continue
        uploaded[name] = r.content
        print(f"  {name}: {len(r.content):,} bytes")
    except Exception as e:
        print(f"  {name}: download failed -- {e}")

print("\nUpload any workbooks you downloaded by hand "
      "(any number, any AMC, any month) -- or cancel if the URLs above covered it:\n")
try:
    uploaded.update(files.upload())
except Exception:
    pass          # cancelled upload box is not an error

# What the database already calls each house. A file resolving to a code that is
# NOT here is a new house -- fine, but worth seeing before it gets created.
#
# Read through amc_summary(), which returns one row per house. A plain
# select on mf_holdings would be capped at 1,000 rows by PostgREST and hand back
# whichever AMC happened to sort first, making every other house look new.
try:
    known = {r["amc"] for r in sb.rpc("amc_summary", {}).execute().data}
except Exception as e:
    print(f"(could not read existing AMC list: {e})")
    known = set()

# ============================================================================
# 3. Parse and check -- nothing is pushed in this loop
# ============================================================================
parsed, skipped = [], []

for fname, blob in uploaded.items():
    amc  = detect_amc(fname)
    freq = detect_frequency(fname)
    print(f"\n{'=' * 70}\n{fname}")

    if not amc:
        print("  SKIPPED -- cannot tell which AMC this is.")
        print("  Fix: set DEFAULT_AMC if this whole run is one house, or add a")
        print("       pattern to AMC_PATTERNS, or")
        print(f'       AMC_OVERRIDE = {{"{fname}": "SHORTCODE"}}')
        skipped.append((fname, "unknown AMC"))
        continue

    print(f"  AMC {amc}   frequency {freq}" +
          ("" if amc in known else "   <-- NEW HOUSE, not in the database yet"))

    try:
        df = parse_workbook(io.BytesIO(blob), amc, freq, fname)
    except Exception as e:
        print(f"  SKIPPED -- parse failed: {e}")
        skipped.append((fname, f"parse error: {e}"))
```

```
        continue

    if df.empty:
        print("  SKIPPED -- parsed to zero rows. Wrong file, or a layout the "
              "parser does not recognise.")
        skipped.append((fname, "zero rows"))
        continue

    ok, n_sch = verify(df, f"  {amc}")
    dates = sorted(d for d in df["portfolio_date"].dropna().unique())
    print(f"  dates           {', '.join(pd.Timestamp(d).strftime('%d-%b-%Y') for d in dates)}")

    if df["portfolio_date"].isna().any():
        print("  SKIPPED -- some rows have no portfolio date.")
        skipped.append((fname, "null dates")); continue

    # The failure that actually corrupts data is the 100x scale decision going
    # the wrong way: every weight and every AUM comes out a hundred times off,
    # and nothing downstream complains. A real portfolio sums near 100 -- an
    # arbitrage or equity-savings book can reach 130 -- so a median outside
    # 20..400 means the scale was misread, not that the fund is unusual.
    med = df.groupby("scheme_name")["pct_to_nav"].sum().median()
    if pd.notna(med) and not (20 < med < 400):
        print(f"  SKIPPED -- scheme weights sum to a median of {med:.2f}, which is "
              "not a percentage. The %NAV column was misread.")
        skipped.append((fname, f"scale wrong (median {med:.1f})")); continue

    # The 100-percent check only means something across a set of schemes. On a
    # one-scheme file it would reject every arbitrage fund, which legitimately
    # sums past 102 -- so warn there instead of skipping. push() flags those rows
    # needs_review either way.
    if n_sch >= 5 and ok / n_sch < 0.70:
        print(f"  SKIPPED -- only {ok}/{n_sch} schemes sum to ~100%. Check the "
              "%NAV column before trusting this file.")
        skipped.append((fname, f"weights off ({ok}/{n_sch})")); continue
    if n_sch < 5 and ok < n_sch:
        print(f"  note: {n_sch - ok} of {n_sch} scheme(s) sum outside 98-102% "
              "(normal for arbitrage and equity-savings) -- flagged, not skipped.")

    parsed.append((fname, amc, df))

# ============================================================================
# 4. Combine per house, then push ONCE
#
# This is not a tidiness choice, it is required. push() deletes by
# (amc, portfolio_date, frequency) before inserting, so pushing 60 single-scheme
# files one after another would have each one wipe the 59 before it and leave a
# single scheme standing. Some AMCs -- Invesco among them -- publish one file per
# scheme rather than one workbook with a sheet per scheme, so this is the normal
# case, not an edge case.
#
# Concatenating first means one delete and one insert per (house, date), which is
# also what makes a re-run safe: it replaces that house's snapshot wholesale.
# ============================================================================
print(f"\n{'=' * 70}\nREADY TO PUSH\n")

by_amc = {}
for fname, amc, df in parsed:
    by_amc.setdefault(amc, []).append(df)

combined = {}
for amc, frames in by_amc.items():
    df = pd.concat(frames, ignore_index=True)
```

```
    # The same scheme arriving in two files at the same date would double every
    # weight. Drop exact duplicates on the natural key before it reaches the DB.
    before = len(df)
    df = df.drop_duplicates(
        subset=["amc", "scheme_name", "portfolio_date", "isin",
                "instrument_name", "market_value_lacs"])
    combined[amc] = df
    dup = before - len(df)
    print(f"  {amc:10s} {df['scheme_name'].nunique():4d} schemes  "
          f"{len(df):7,} rows   from {len(frames)} file(s)"
          + (f"   ({dup:,} duplicate rows dropped)" if dup else ""))

if skipped:
    print("\nSKIPPED")
    for fname, why in skipped:
        print(f"  {why:24s} {fname}")

if not combined:
    print("\nNothing to push.")
else:
    print()
    for amc, df in combined.items():
        push(df)

    print(f"\n{'=' * 70}")
    print("Pushed. Now run sql/after_load.sql in Supabase -- until you do, the")
    print("new funds are in mf_holdings but invisible to every market-wide")
    print("query, because the analytical layer is materialised.")
```

Appendix C · invesco.py — Invesco (API se apne aap)

Sirf MONTH (aur January mein YEAR) badalna. Baaki khud ho jaata hai.

```
"""
Invesco -> mf_holdings, off their own API

The disclosure page is a Next.js app that builds its file list client-side, so
there is nothing to scrape from the HTML. The contract is written in their JS
bundle, and this uses it exactly as their own page does:

    GET /api/ClassificationCompleteMonthlyHoldings
        -> [{FunClassificationValue: "equity", ...}, ...]   the category slugs

    GET /api/CompleteMonthlyHoldings?year=YYYY&classification=<slug>
        -> one record per scheme, with a column PER MONTH:
           JanUrl / JanName, FebUrl / FebName, ... DecUrl / DecName

That per-month column shape is the thing worth knowing. There is no DocumentUrl
field, which is why a generic "find the download link" walk over this response
comes back empty.

    Cell 1 : colab/parser.py
    Cell 2 : this file
    Then   : sql/after_load.sql in Supabase

Invesco publishes one workbook per scheme, so a month is ~60 files across seven
categories. They are concatenated and pushed once -- push() deletes by
(amc, portfolio_date, frequency), so pushing them individually would have each
file wipe the ones before it.
"""

import subprocess, sys
subprocess.run([sys.executable, "-m", "pip", "install", "-q",
                "requests", "pandas", "openpyxl", "xlrd", "supabase"], check=False)

import io, re, json, time
import requests, pandas as pd
from supabase import create_client

SUPABASE_URL = "https://ulunrpbayvlazzxpqrpj.supabase.co"
SUPABASE_KEY = "PASTE_SERVICE_ROLE_KEY_HERE"      # service_role

AMC       = "Invesco"
FREQUENCY = "monthly"
YEAR      = 2026
MONTH     = "Jun"        # three-letter prefix: Jan Feb Mar ... Dec

HOST = "https://www.invescomutualfund.com"

S = requests.Session()
S.headers.update({
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 "
                  "(KHTML, like Gecko) Chrome/126.0 Safari/537.36",
    "Accept": "application/json, text/plain, */*",
    "Content-Type": "application/json",
    "Referer": f"{HOST}/literature-forms/monthly-holdings/equity",
})
```

```
# ------------------------------------------------------------------------
# Key guard -- a key from another project fails with a bare "401 Invalid API
# key" only at the push, after every file has been fetched and parsed. It has
# happened twice here. Both the project ref and the role are in the JWT.
# ------------------------------------------------------------------------
def check_key(url, key):
    import base64
    if key.startswith("PASTE_"):
        raise SystemExit("SUPABASE_KEY is still the placeholder.")
    body = key.split(".")[1]; body += "=" * (-len(body) % 4)
    c = json.loads(base64.urlsafe_b64decode(body))
    want = url.split("//")[1].split(".")[0]
    print(f"key: role={c.get('role')}  project={c.get('ref')}  (expected {want})")
    if c.get("ref") != want:
        raise SystemExit(f"WRONG PROJECT -- key belongs to '{c.get('ref')}'. Nothing pushed.")
    if c.get("role") != "service_role":
        raise SystemExit(f"This is the '{c.get('role')}' key; writes need service_role.")


check_key(SUPABASE_URL, SUPABASE_KEY)

for _fn in ("parse_workbook", "verify", "make_push"):
    if _fn not in globals():
        raise SystemExit(f"'{_fn}' undefined -- run the parser.py cell first.")

sb   = create_client(SUPABASE_URL, SUPABASE_KEY)
push = make_push(sb)

MONTH_KEY = re.compile(r"^(jan|feb|mar|apr|may|jun|jul|aug|sep|oct|nov|dec)url$", re.I)


# ============================================================================
# 1. Categories, then one call per category
# ============================================================================
print("\n" + "=" * 74)
print("1. API")
print("=" * 74)

cls = S.get(f"{HOST}/api/ClassificationCompleteMonthlyHoldings", timeout=90).json()
slugs = [c["FunClassificationValue"] for c in cls
         if c.get("FunClassificationValue", "").lower() != "select"]
print(f"  {len(slugs)} categories: {', '.join(slugs)}")

items, shape_shown = {}, False

for slug in slugs:
    u = f"{HOST}/api/CompleteMonthlyHoldings?year={YEAR}&classification={slug}"
    try:
        r = S.get(u, timeout=120)
        recs = r.json() if r.ok else []
    except Exception as e:
        print(f"  {slug:22s} failed: {e}")
        continue
    if not isinstance(recs, list):
        recs = [recs]

    # Print the first record's keys once. If Invesco renames a column, this is
    # where it shows -- far better than a silent zero.
    if recs and not shape_shown:
        shape_shown = True
        print(f"\n  record keys: {', '.join(list(recs[0].keys())[:24])}\n")
```

```
    hit = 0
    for rec in recs:
        if not isinstance(rec, dict):
            continue
        for k, v in rec.items():
            m = MONTH_KEY.match(k)
            if not m or not isinstance(v, str) or not v.strip():
                continue
            if m.group(1).lower() != MONTH.lower()[:3]:
                continue
            if not re.search(r"\.(xlsx|xls)(\?|$)", v, re.I):
                continue
            url  = v if v.startswith("http") else HOST + "/" + v.lstrip("/")
            name = rec.get(k[:3] + "Name") or rec.get("SchemeName") or url
            items.setdefault(url, {"url": url, "name": str(name), "cat": slug})
            hit += 1
    print(f"  {slug:22s} {len(recs):4d} record(s)  {hit:4d} {MONTH} workbook(s)")

print(f"\n{len(items)} unique workbook(s) for {MONTH}-{YEAR}")
if not items:
    raise SystemExit(
        f"Nothing for {MONTH}-{YEAR}. Either that month is not published yet, or a "
        "column was renamed -- check the 'record keys' line above.")


# ============================================================================
# 2. Download, parse, check
# ============================================================================
print("\n" + "=" * 74)
print("2. DOWNLOAD + PARSE")
print("=" * 74)

frames, skipped = [], []

for i, it in enumerate(sorted(items.values(), key=lambda x: x["name"]), 1):
    fname = it["url"].rsplit("/", 1)[-1].split("?")[0]
    try:
        r = S.get(it["url"], timeout=180)
    except Exception as e:
        skipped.append((fname, f"download failed: {e}")); continue
    if r.status_code != 200:
        skipped.append((fname, f"http {r.status_code}")); continue
    # A moved file comes back as an HTML error page with status 200, which would
    # parse to zero rows and read like a parser fault.
    if r.content[:200].lstrip()[:1] == b"<":
        skipped.append((fname, "served HTML, not a workbook")); continue

    try:
        df = parse_workbook(io.BytesIO(r.content), AMC, FREQUENCY, fname)
    except Exception as e:
        skipped.append((fname, f"parse error: {e}")); continue
    if df.empty:
        skipped.append((fname, "zero rows")); continue
    if df["portfolio_date"].isna().any():
        skipped.append((fname, "null portfolio date")); continue

    # The one failure that corrupts rather than omits: the 100x scale decision
    # going the wrong way makes every weight and every AUM a hundred times off,
    # and nothing downstream complains. A real book sums near 100; arbitrage can
    # reach 130. Outside 20..400 the %NAV column was misread.
    med = df.groupby("scheme_name")["pct_to_nav"].sum().median()
    if pd.notna(med) and not (20 < med < 400):
```

```
        skipped.append((fname, f"scale wrong (median {med:.1f})")); continue

    frames.append(df)
    print(f"  [{i:3d}/{len(items)}] {len(df):5,} rows  "
          f"{sorted(df['scheme_name'].unique())[0][:56]}")
    time.sleep(0.2)

if skipped:
    print(f"\n{len(skipped)} skipped")
    for n, why in skipped:
        print(f"  {why:34s} {n}")

if not frames:
    raise SystemExit("\nNothing parsed. Nothing pushed.")


# ============================================================================
# 3. One push for the whole house
# ============================================================================
df = pd.concat(frames, ignore_index=True)
before = len(df)
df = df.drop_duplicates(subset=["amc", "scheme_name", "portfolio_date", "isin",
                                "instrument_name", "market_value_lacs"])
if before != len(df):
    print(f"\n{before - len(df):,} duplicate row(s) dropped")

print()
ok, n_sch = verify(df, AMC)
print("  dates           " + ", ".join(sorted(
    pd.Timestamp(d).strftime("%d-%b-%Y") for d in df["portfolio_date"].dropna().unique())))
print(f"  files parsed    {len(frames)} of {len(items)}")

if n_sch >= 5 and ok / n_sch < 0.70:
    raise SystemExit(f"\nSTOPPED -- only {ok}/{n_sch} schemes sum to ~100%.")

print()
push(df)

print("\n" + "=" * 74)
print("Pushed. Now run sql/after_load.sql in Supabase, then confirm:")
print("  select count(distinct scheme_name) from mf_holdings where amc = 'Invesco';")
```

Appendix D · jioblackrock.py — JioBlackRock (server action se)

Sirf TARGET (YYYY-MM) badalna, ya None chhodo to latest aa jaata hai.

```
"""
JioBlackRock -> mf_holdings, off their own Strapi-backed API

The disclosure page (https://www.jioblackrockamc.com/statutory-disclosure/
disclosures/monthly-portfolio-disclosure) is a Next.js app whose file list is
built client-side by a SERVER ACTION, so there is no .xlsx link to scrape. The
action returns JSON where each record has file.url pointing at a public CDN
(jioinvest.cdn.jio.com). We call the action directly, exactly as the page does.

    Cell 1 : colab/parser.py
    Cell 2 : this file
    Then   : sql/after_load.sql in Supabase

CHANGE EACH MONTH: only TARGET (the YYYY-MM you want). Leave it None to auto-pick
the latest month the site has published. The action id is re-read from the page
on every run, so a site redeploy does not break it.
"""

import subprocess, sys
subprocess.run([sys.executable, "-m", "pip", "install", "-q",
                "requests", "pandas", "openpyxl", "xlrd", "supabase"], check=False)

import io, re, json, time
import requests, pandas as pd
from supabase import create_client

SUPABASE_URL = "https://ulunrpbayvlazzxpqrpj.supabase.co"
SUPABASE_KEY = "PASTE_SERVICE_ROLE_KEY_HERE"      # service_role

AMC       = "JioBR"       # short code -- keep it EXACTLY this every month
FREQUENCY = "monthly"
TARGET    = None          # "2026-06" style YYYY-MM, or None = latest available

PAGE = "https://www.jioblackrockamc.com/statutory-disclosure/disclosures/monthly-portfolio-
disclosure"

S = requests.Session()
S.headers.update({"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
                  "AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0 Safari/537.36"})


# ---- key guard: a wrong-project key fails only at push, after all work ----
def check_key(url, key):
    import base64
    if key.startswith("PASTE_"):
        raise SystemExit("SUPABASE_KEY placeholder hai.")
    body = key.split(".")[1]; body += "=" * (-len(body) % 4)
    c = json.loads(base64.urlsafe_b64decode(body))
    want = url.split("//")[1].split(".")[0]
    print(f"key: role={c.get('role')}  project={c.get('ref')}  (expected {want})")
    if c.get("ref") != want:
        raise SystemExit(f"WRONG PROJECT -- key '{c.get('ref')}' ki hai. Kuch push nahi hua.")
    if c.get("role") != "service_role":
        raise SystemExit(f"'{c.get('role')}' key hai; likhne ke liye service_role chahiye.")
```

```
check_key(SUPABASE_URL, SUPABASE_KEY)
for _fn in ("parse_workbook", "verify", "make_push"):
    if _fn not in globals():
        raise SystemExit(f"'{_fn}' undefined -- pehle parser.py wala cell chalao.")

sb = create_client(SUPABASE_URL, SUPABASE_KEY)
push = make_push(sb)


# ---- 1. call the server action, read the file list ----
print("\n" + "=" * 60 + "\n1. FILE LIST (server action)")
html = S.get(PAGE, timeout=120).text
m = re.search(r'createServerReference\)\("([a-f0-9]{20,})"[^)]*"getDisclosureL3Data"', html)
action = m.group(1) if m else "70515daed85b22e9b12b90e52bedc499c23dd3859f"

r = S.post(PAGE, timeout=120,
           headers={"Next-Action": action, "Content-Type": "text/plain;charset=UTF-8",
                    "Accept": "text/x-component"},
           data=b'["monthly-portfolio-disclosure"]')
i = r.text.find('{"data":')
if i < 0:
    raise SystemExit("data nahi mila -- action badla hoga, invesco_probe style se dobara dekho")
records = json.JSONDecoder().raw_decode(r.text[i:])[0]["data"]
print(f"  {len(records)} document(s)")

files = []
for rec in records:
    f = rec.get("file") or {}
    url = (f.get("url") or "").replace("\\/", "/")
    if re.search(r"\.(xlsx|xls)$", url, re.I):
        files.append({"url": url, "date": rec.get("date", "")})

buckets = {}
for it in files:
    ym = it["date"][:7] if re.match(r"\d{4}-\d{2}", it["date"]) else "?"
    buckets[ym] = buckets.get(ym, 0) + 1
print("  available (YYYY-MM):")
for ym in sorted(buckets, reverse=True):
    print(f"    {ym}  {buckets[ym]:3d}")

target = TARGET or max(b for b in buckets if b != "?")
files = [it for it in files if it["date"].startswith(target)]
print(f"\n  TARGET = {target}  ->  {len(files)} file(s)")
if not files:
    raise SystemExit("Is month par kuch nahi -- TARGET badlo.")


# ---- 2. download, parse, check ----
print("\n" + "=" * 60 + "\n2. DOWNLOAD + PARSE")
frames, skipped = [], []
for k, it in enumerate(sorted(files, key=lambda x: x["url"]), 1):
    fname = it["url"].rsplit("/", 1)[-1]
    try:
        rr = S.get(it["url"], timeout=180)
    except Exception as e:
        skipped.append((fname, f"download: {e}")); continue
    if rr.status_code != 200 or rr.content[:200].lstrip()[:1] == b"<":
        skipped.append((fname, "bad response")); continue
    try:
        df = parse_workbook(io.BytesIO(rr.content), AMC, FREQUENCY, fname)
    except Exception as e:
```

```
        skipped.append((fname, f"parse: {e}")); continue
    if df.empty or df["portfolio_date"].isna().any():
        skipped.append((fname, "empty/nulldate")); continue
    med = df.groupby("scheme_name")["pct_to_nav"].sum().median()
    if pd.notna(med) and not (20 < med < 400):
        skipped.append((fname, f"scale {med:.1f}")); continue
    frames.append(df)
    print(f"  [{k:2d}/{len(files)}] {len(df):4,} rows  {sorted(df['scheme_name'].unique())[0]
[:52]}")
    time.sleep(0.2)

if skipped:
    print(f"\n{len(skipped)} skipped")
    for n, why in skipped: print(f"  {why:22s} {n}")
if not frames:
    raise SystemExit("\nKuch parse nahi hua.")


# ---- 3. one push ----
df = pd.concat(frames, ignore_index=True).drop_duplicates(
    subset=["amc", "scheme_name", "portfolio_date", "isin", "instrument_name",
"market_value_lacs"])
print()
verify(df, AMC)
print()
push(df)
print("\nPushed. Ab Supabase mein sql/after_load.sql chalao, phir:")
print("  select count(distinct scheme_name) from mf_holdings where amc = 'JioBR';")
```

Appendix E · after_load.sql — Supabase mein har load ke baad chalao

Supabase → SQL Editor mein poora paste karke Run. Ye naye funds ko website ke liye taiyaar karta hai. Bina iske naye funds nahi dikhenge.

```
-- ============================================================================
-- Run this after every push to mf_holdings -- new AMC, new month, re-load.
-- ----------------------------------------------------------------------------
-- Everything downstream of scheme_alias is a view or a function, so it picks
-- up new data on its own. scheme_alias is a real table (it has to survive the
-- monthly delete-then-insert on mf_holdings), so it is the one thing that
-- needs refreshing by hand.
--
-- PART A is the normal case and is usually all you need.
-- PART B is only for funds AMFI has not listed yet.
-- ============================================================================


-- ============================================================================
-- PART A -- after any load
-- ============================================================================

-- ----------------------------------------------------------------------------
-- A1. Map the new scheme names. Idempotent -- existing aliases are untouched,
--     including the manual ones, because of the ON CONFLICT.
-- ----------------------------------------------------------------------------
insert into scheme_alias (amc, source_name, k, matched_by)
select distinct h.amc, h.scheme_name, b.k, 'auto'
from mf_holdings h
join scheme_base b on b.k = norm_key(h.scheme_name)
on conflict do nothing;


-- ----------------------------------------------------------------------------
-- A2. What did NOT map. These schemes hold data but will not appear in the
--     dashboard's search results as "Portfolio", and holdings_by_code will
--     not reach them.
--
--     Expect roughly 35 rows even on a clean run: 23 close-ended (SBI/ICICI
--     FMP series, Nippon Quarterly Interval) that AMFI's master genuinely does
--     not carry and never will, plus a dozen funds too new to be listed.
--     Anything beyond that is worth looking at.
-- ----------------------------------------------------------------------------
select h.amc,
       h.scheme_name,
       count(*)                as rows,
       max(h.portfolio_date)   as latest
from mf_holdings h
left join scheme_alias a
       on a.amc = h.amc and a.source_name = h.scheme_name
where a.k is null
group by 1, 2
order by 1, 2;


-- ----------------------------------------------------------------------------
-- A3. Cross-AMC guard. A wrong alias now fans out across every plan variant
--     of the fund, so a single bad match pollutes far more than one row.
```

```
--
--     ADD ANY NEW AMC to the map below, otherwise it is silently skipped and
--     never checked. The short codes in mf_holdings ('ABSL', '360ONE') do not
--     prefix-match master's full names, which is why the map is explicit.
-- ------------------------------------------------------------------------
with amc_map(amc, prefix) as (values
    ('ABSL','aditya birla sun life'), ('HDFC','hdfc'),   ('SBI','sbi'),
    ('ICICI','icici prudential'),     ('Nippon','nippon india'),
    ('Kotak','kotak'),                ('DSP','dsp'),     ('Motilal','motilal oswal'),
    ('Bandhan','bandhan'),            ('Mirae','mirae asset'), ('Axis','axis'),
    ('Tata','tata'),                  ('Quant','quant '), ('Quantum','quantum'),
    ('Abakkus','abakkus'),            ('360ONE','360 one')
    -- ('Invesco','invesco'), ('HSBC','hsbc'), ('UTI','uti'), ...
)
select distinct p.amc, p.scheme_name, p.master_amc, p.master_scheme_name, p.matched_by
from v_scheme_plans p
join amc_map m on m.amc = p.amc
where lower(p.master_amc) not like m.prefix || '%'
  -- Nippon India was formerly Reliance Mutual Fund. Correct, not an error.
  and not (p.amc = 'Nippon' and lower(p.master_amc) like 'reliance%')
order by 1, 2;


-- ------------------------------------------------------------------------
-- A4. Rebuild the analytical layer. REQUIRED -- these are materialised, so new
--     holdings are invisible to every market-wide query until they refresh.
--
--     Order matters: mv_current reads mv_security, mv_fund_stats reads
--     mv_current.
-- ------------------------------------------------------------------------
refresh materialized view mv_security;
refresh materialized view mv_current;
refresh materialized view mv_previous;
refresh materialized view mv_fund_stats;

-- REFRESH resets the planner statistics too, so re-analyze or queries that were
-- fast yesterday start timing out.
--
-- NOTE: refresh is enough for NEW DATA. It was NOT enough after changing the
-- body of asset_class() -- mv_current kept serving the old classification
-- through repeated refreshes. If you edit that function, drop and recreate
-- mv_current and mv_fund_stats (sql/engine_core.sql) rather than refreshing.
analyze mv_security;
analyze mv_current;
analyze mv_previous;
analyze mv_fund_stats;

-- Only after a colab/nav_monthly_ingest.py run, not after a holdings load --
-- mv_fund_returns reads NAV history, which a portfolio file does not carry.
-- Harmless to run anyway; it just rebuilds the same numbers.
refresh materialized view mv_fund_returns;
analyze mv_fund_returns;


-- ------------------------------------------------------------------------
-- A5. Confirm the dashboard sees the new funds.
-- ------------------------------------------------------------------------
select amc,
       count(*)                  as funds,
       sum(plan_count)           as plan_codes,
       max(latest_portfolio_date) as latest_date
from v_fund_picker
group by 1
```

```
order by funds desc;

-- Nothing else to do. v_scheme_plans, v_fund_picker, v_holdings_all_plans,
-- search_schemes, holdings_by_code and every holdings_* RPC read through to
-- the new rows immediately. The dashboard HTML does not change.


-- ============================================================================
-- PART B -- only when a fund is too new for AMFI's master
-- ----------------------------------------------------------------------------
-- Symptom: a scheme stays in the A2 list even though the fund clearly exists.
-- Its holdings loaded fine, but schemes_master has no row to map onto, so it
-- has no scheme_code and cannot be searched.
--
-- Refresh schemes_master from AMFI first (colab/amfi_nav.py parses the same
-- file), then rebuild the derived layer below.
-- ============================================================================

-- ----------------------------------------------------------------------------
-- B1. Rebuild scheme_base IN PLACE.
--
--     Do NOT drop and recreate it. v_holdings, v_scheme_plans, v_fund_picker,
--     v_holdings_all_plans and scheme_key_meta all depend on it now, and a
--     DROP would take the whole stack with it. TRUNCATE + INSERT keeps every
--     dependent object and every index intact.
-- ----------------------------------------------------------------------------
begin;

truncate scheme_base;

insert into scheme_base (k, base_name, scheme_code, plan_type, amc_name, category, full_name)
select norm_key(base_scheme(scheme_name)),
       base_scheme(scheme_name),
       scheme_code, plan_type, amc_name, category,
       scheme_name
from schemes_master;

commit;

-- B2. The metadata fallback is materialised, so it does not follow along.
refresh materialized view scheme_key_meta;

-- B3. Now re-run A1 to pick up the newly listed funds, then A2 to see what is
--     left. Some of the previously unmapped schemes should disappear.


-- ============================================================================
-- QUICK STATUS -- run any time
-- ============================================================================
select (select count(*) from mf_holdings)                          as holdings_rows,
       (select count(distinct (amc, scheme_name)) from mf_holdings) as schemes_loaded,
       (select count(*) from scheme_alias)                          as mapped,
       (select count(*) from v_fund_picker)                         as funds_searchable,
       (select count(*) from v_scheme_plans)                        as plan_codes,
       (select count(*) from scheme_base)                           as master_rows;
```

Appendix F · nav_monthly_ingest.py — optional (Leaders / AMC-median returns)

Sirf tab chalao jab Leaders tab ya AMC-median returns refresh karne ho. Compare ke returns ko iski zaroorat nahi. Iske baad Supabase mein: refresh materialized view mv_fund_returns; analyze mv_fund_returns;

```
"""
Month-end NAV for every fund with a loaded portfolio  ->  scheme_nav_monthly

Feeds return_leaderboard() and fund_returns_stored() (sql/returns_and_caps.sql).

Why monthly, and why one code per fund:
  Daily NAV for all 4,060 plan codes would be millions of rows and will not fit
  the free tier. Monthly is enough for CAGR and for ranking. The per-fund page
  still pulls DAILY history live from api.mfapi.in for its chart and risk
  metrics, so nothing is lost there.

Why Direct-Growth:
  It is the standard comparison basis -- no distributor commission -- so a
  leaderboard is not reordered by which plan happened to get stored.

Expect ~1,100 funds x ~120 months = roughly 130k rows.

Run sql/returns_and_caps.sql first (it creates the table).
"""

# ============================================================================
# CELL 1 -- pick one representative scheme_code per fund
# ============================================================================
# Self-installing: a fresh Colab session has no supabase client, and a
# commented-out !pip line is a trap that fails on the import below.
import subprocess, sys
subprocess.run([sys.executable, "-m", "pip", "install", "-q",
                "requests", "pandas", "supabase"], check=False)

import re, time, json, requests, pandas as pd
from concurrent.futures import ThreadPoolExecutor
from supabase import create_client

# service_role, not anon: scheme_nav_monthly has RLS with a read-only policy,
# so an anon key cannot insert.
SUPABASE_URL = "https://ulunrpbayvlazzxpqrpj.supabase.co"
SUPABASE_KEY = "PASTE_SERVICE_ROLE_KEY_HERE"

sb = create_client(SUPABASE_URL, SUPABASE_KEY)

# PostgREST caps a single response at 1,000 rows. v_scheme_plans holds 4,060,
# so an unpaged read silently returned the first 1,000 -- which grouped down to
# 368 funds, and the other 750+ were never fetched. No error, just missing data.
def fetch_all(tbl, cols, step=1000):
    out, start = [], 0
    while True:
        chunk = (sb.table(tbl).select(cols)
                   .range(start, start + step - 1).execute().data)
        out += chunk
        print(f"  {tbl}: {len(out):,} rows")
        if len(chunk) < step:
```

```
            return out
        start += step

plans = pd.DataFrame(fetch_all("v_scheme_plans",
                               "amc,scheme_name,scheme_code,plan_name,plan_type"))
print(f"{len(plans):,} plan rows")
assert len(plans) > 3000, f"only {len(plans)} rows returned -- paging is not working"

def score(name):
    """Lower is better. Direct + Growth wins; IDCW and Regular are penalised."""
    n = (name or "").lower()
    s = 0
    if "direct" not in n:            s += 10
    if "growth" not in n:            s += 5
    if re.search(r"idcw|dividend|payout|reinvest|bonus", n): s += 20
    return s

plans["s"] = plans["plan_name"].map(score)
pick = (plans.sort_values(["amc", "scheme_name", "s", "scheme_code"])
             .groupby(["amc", "scheme_name"], as_index=False).first())

print(f"{len(pick):,} funds -> one code each")
print(pick[["amc", "scheme_name", "plan_name", "scheme_code"]].head(10).to_string(index=False))

# Sanity: most picks should be Direct Growth. A high count here means the
# plan_name text is not what the scorer expects -- inspect before running cell 2.
bad = pick[pick.s > 5]
print(f"\n{len(bad)} funds had no Direct-Growth plan available")
print(bad[["amc", "scheme_name", "plan_name"]].head(15).to_string(index=False))


# ============================================================================
# CELL 2 -- fetch history and downsample to month-end
# ============================================================================
MFAPI = "https://api.mfapi.in/mf/"
sess = requests.Session()
sess.headers["User-Agent"] = "Mozilla/5.0"

def fetch(code):
    code = str(code)
    for attempt in range(3):
        try:
            r = sess.get(MFAPI + code, timeout=45)
            if r.status_code != 200:
                return code, None, f"http {r.status_code}"
            d = r.json().get("data")
            if not d:
                return code, None, "empty"
            df = pd.DataFrame(d)
            df["month_end"] = pd.to_datetime(df["date"], format="%d-%m-%Y", errors="coerce")
            df["nav"] = pd.to_numeric(df["nav"], errors="coerce")
            df = df.dropna(subset=["month_end", "nav"])
            df = df[df["nav"] > 0]
            if df.empty:
                return code, None, "no valid rows"
            # last observation of each calendar month -- NAV is not published on
            # every date, so the true month end is whatever the fund last quoted
            df = (df.sort_values("month_end")
                    .groupby(df["month_end"].dt.to_period("M"), as_index=False)
                    .last()[["month_end", "nav"]])
            df["scheme_code"] = code
            return code, df, None
        except Exception as e:
```

```
            if attempt == 2:
                return code, None, str(e)[:60]
            time.sleep(1.5 * (attempt + 1))

codes = sorted(set(pick["scheme_code"].astype(str)))   # same code can back two funds
frames, failed = [], []

with ThreadPoolExecutor(max_workers=12) as ex:
    for i, (code, df, err) in enumerate(ex.map(fetch, codes), 1):
        if df is None:
            failed.append((code, err))
        else:
            frames.append(df)
        if i % 100 == 0:
            print(f"  {i}/{len(codes)}  ok={len(frames)}  failed={len(failed)}")

nav = pd.concat(frames, ignore_index=True) if frames else pd.DataFrame()
print(f"\nfunds fetched   {len(frames):,}")
print(f"funds failed    {len(failed):,}")
print(f"rows            {len(nav):,}")
if len(nav):
    print(f"date range      {nav.month_end.min().date()} -> {nav.month_end.max().date()}")
    print(f"months per fund median {nav.groupby('scheme_code').size().median():.0f}")
print("\nfailures:", failed[:15])


# ============================================================================
# CELL 3 -- push
# ============================================================================
up = nav.copy()
up["month_end"] = up["month_end"].dt.strftime("%Y-%m-%d")
up["nav"] = up["nav"].round(4)

# One scheme_code can arrive under two funds when two scheme names resolve to
# the same key in scheme_alias, so the same series gets fetched twice and the
# (scheme_code, month_end) primary key rejects the second copy mid-push.
up = up[["scheme_code", "month_end", "nav"]].drop_duplicates(
        subset=["scheme_code", "month_end"], keep="last")
print(f"{len(nav):,} rows -> {len(up):,} after dedupe")
print(f"unique funds: {up.scheme_code.nunique():,}")

recs = json.loads(up.to_json(orient="records"))

# Full replace. upsert would work too, but a clean wipe avoids leaving rows
# behind for funds that dropped out of the picker since the last run.
sb.table("scheme_nav_monthly").delete().neq("scheme_code", "").execute()
for i in range(0, len(recs), 1000):
    sb.table("scheme_nav_monthly").insert(recs[i:i + 1000]).execute()
    if (i // 1000) % 20 == 0:
        print(f"  {i:,}/{len(recs):,}")
print(f"pushed {len(recs):,}")


# ============================================================================
# CELL 4 -- verify in Supabase
# ============================================================================
# select count(*) rows, count(distinct scheme_code) funds,
#        min(month_end), max(month_end)
# from scheme_nav_monthly;
#
# select * from return_leaderboard(36, 'Large Cap Fund', null, 20);
#
# Cross-check one fund against its own page: the 3Y CAGR from stored monthly
```

```
# NAV should land within a few basis points of the daily figure the dashboard
# computes live. A large gap means the month-end downsampling picked up a
# stale quote.
```